

Universidad Tecnológica Nacional  
Facultad Regional San Rafael  
Ingeniería en Sistemas de Información

# Guía de desarrollo

TPI Software Metrics & Estimation · Ingeniería y Calidad de Software 2026

**Cómo vamos a trabajar, qué construye cada uno, para qué sirve  
y ejemplos explicados paso a paso**

## Equipo

López, Juan Pablo — Agile Enabler y Product Builder  
Piastrellini, Mariano — Product Builder  
Bravo, Carolina — Product Builder

Versión 1.1 · 26/09/2026 · Complementa el **Plan de trabajo** (backlog e historias completas).

## 0. Decisiones tomadas (cambios respecto al plan v1.0)

| Tema                     | Decisión                                                                                                | Motivo                                                                                                                                                                                                                |
|--------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Agile Enabler            | <b>Juan Pablo</b>                                                                                       | Ya administra el repositorio y el tablero.                                                                                                                                                                            |
| Duración de los sprints  | <b>1 semana</b> , de lunes a domingo                                                                    | Las clases terminan a principios de noviembre.                                                                                                                                                                        |
| Usuarios                 | <b>Multiusuario con login</b> (Supabase Auth)                                                           | Cada integrante vota en Planning Poker y carga sus horas desde su cuenta.                                                                                                                                             |
| Interfaz                 | <b>Go + HTMX + Bootstrap 5</b> (sin React/Next.js)                                                      | Nadie del equipo programó antes: un solo lenguaje (Go) en vez de dos. Bootstrap da una estética prolija con componentes listos (tarjetas, menús, modales) y HTMX permite pantallas dinámicas sin escribir JavaScript. |
| ¿Quién hace la interfaz? | <b>Cada uno hace las pantallas de sus funcionalidades</b> ; Carolina arma el diseño base y el dashboard | Así la carga queda pareja y cada uno puede mostrar su funcionalidad completa en la demo.                                                                                                                              |
| Esqueleto del proyecto   | Juan Pablo lo arma <b>con Claude</b> en el Sprint 0, ya funcionando y con un ejemplo completo           | El Sprint 1 arranca sobre una base que compila, con tests y deploy andando.                                                                                                                                           |

### Calendario (a confirmar la fecha exacta de la presentación final)

| Sprint                      | Fechas                | Objetivo                                                                         | Review                   |
|-----------------------------|-----------------------|----------------------------------------------------------------------------------|--------------------------|
| 0 · Preparación             | lun 28/09 – dom 04/10 | Entorno, tablero, backlog, aprender lo básico, esqueleto andando                 | Tablero y backlog listos |
| 1 · MVP                     | lun 05/10 – dom 11/10 | Versión funcional básica: proyectos, backlog, sprints, horas y primeras métricas | Demo en Vercel           |
| 2 · Interfaz                | lun 12/10 – dom 18/10 | Login, pantallas completas, cierre de sprint, Planning Poker, defectos           | Demo en Vercel           |
| 3 · Funcionalidad y calidad | lun 19/10 – dom 25/10 | Dashboard con gráficos, métricas completas, reportes, cobertura                  | Demo en Vercel           |
| 4 · Cierre                  | lun 26/10 – dom 01/11 | Exportar PDF, documentación, manual, informe y presentación                      | Review final             |

#### Plan de recorte (si un sprint se atrasa)

- Se recorta en este orden, sin tocar los requerimientos mínimos: HU-10 (filtros), orden manual de HU-09 (queda sólo por prioridad), HU-25 (tareas: el esfuerzo se registra sólo por historia, la consigna dice "historia • tarea"), edición de registros en HU-26, TEC-06 (tests de integración).
- Si en la Retro del Sprint 1 vemos que la velocidad real es menor, el Agile Enabler replanifica: nunca se estira un sprint.

# 1. Cómo funciona una aplicación web (explicado desde cero)

Antes de escribir código conviene entender **qué pasa cuando alguien usa la app**. Tomemos un ejemplo: el usuario completa el formulario "Nuevo proyecto" y aprieta **Crear**.

| # | Capa             | Qué hace en el ejemplo                                                                                 | Analogía (restaurante)    | Carpeta          |
|---|------------------|--------------------------------------------------------------------------------------------------------|---------------------------|------------------|
| 1 | Navegador + HTMX | Envía los datos del formulario al servidor y, con la respuesta, reemplaza sólo un pedazo de la página. | El cliente hace el pedido | web/templates    |
| 2 | Router           | Mira la dirección (POST /proyectos) y decide qué función la atiende.                                   | El recepcionista          | internal/http    |
| 3 | Handler          | Lee los campos del formulario, llama al servicio y arma la respuesta HTML.                             | El mozo                   | internal/http    |
| 4 | Servicio         | Coordina el caso de uso: valida, verifica que el nombre no exista, guarda.                             | El jefe de cocina         | internal/service |
| 5 | Dominio          | Las <b>reglas de negocio</b> : nombre de 3 a 100 letras, fin después del inicio, estado inicial.       | La receta                 | internal/domain  |
| 6 | Repositorio      | Guarda y lee datos de la base, con SQL.                                                                | La despensa               | internal/store   |
| 7 | Base de datos    | PostgreSQL en Supabase: tablas con los datos.                                                          | La heladera               | Supabase         |

**Por qué separar en capas:** el dominio no sabe nada de la web ni de la base de datos, así que sus tests son simples y rápidos (eso es lo que más se evalúa con TDD). Si mañana cambiamos Supabase por otra base, sólo cambia la despensa, no la receta.

## Tecnologías y para qué sirve cada una

| Herramienta   | Para qué la usamos                                                                                        |
|---------------|-----------------------------------------------------------------------------------------------------------|
| Go            | El lenguaje de todo el backend: reglas, cálculos, validaciones, servidor web.                             |
| go test       | Ejecuta los tests unitarios. Es parte de Go, no hay que instalar nada.                                    |
| godog         | Ejecuta los escenarios BDD escritos en español (Dado / Cuando / Entonces) como tests.                     |
| chi           | Router: asocia cada dirección (URL) con su handler.                                                       |
| html/template | Plantillas HTML de Go: páginas con huecos que se completan con datos.                                     |
| HTMX          | Atributos como hx-post en el HTML que hacen pedidos al servidor sin recargar la página.                   |
| Bootstrap 5   | Estilos y componentes visuales listos (botones, tarjetas, tablas, formularios).                           |
| Chart.js      | Gráficos del dashboard (barras, líneas, tortas).                                                          |
| Supabase      | Base de datos PostgreSQL en la nube + login de usuarios. Plan gratuito.                                   |
| pgx           | Librería de Go para hablar con PostgreSQL.                                                                |
| Vercel        | Publica la app en internet. Cada push genera una versión nueva automáticamente.                           |
| Git + GitHub  | Historial de cambios, trabajo en paralelo con ramas, revisión de código con Pull Requests, tablero Scrum. |

## 2. Preparar la PC (Sprint 0, una sola vez)

Todos los pasos son para Windows. Abrir **PowerShell** y ejecutar de a una línea:

### 1) Instalar herramientas

```
winget install --id GoLang.Go -e          # instala Go
winget install --id Git.Git -e           # instala Git
winget install --id Microsoft.VisualStudioCode -e
winget install --id GitHub.cli -e        # CLI de GitHub (gh)
```

Cerrar y volver a abrir PowerShell para que tome los cambios. En VS Code, instalar la extensión **Go** (de Google): da autocompletado, formateo automático y un botón para correr cada test.

### 2) Verificar

```
go version                                # debe mostrar go1.xx
git --version
gh auth login                             # iniciar sesión con la cuenta de GitHub de cada uno
go install github.com/cucumber/godog/cmd/godog@latest
```

### 3) Clonar el repositorio y probar que todo anda

```
git clone https://github.com/<cuenta-institucional>/<repo>.git
cd <repo>
git config user.name "Apellido, Nombre"
git config user.email "<email-de-tu-cuenta-de-GitHub>"
copy .env.example .env                    # y completar las claves que pasa Juan Pablo (nunca se suben a GitHub)
go test ./...                             # corre todos los tests
go run ./cmd/server                       # levanta la app en http://localhost:8080
```

## 2.1 Qué aprender antes del Sprint 1 (tareas del Sprint 0)

No hace falta saber todo: se aprende historia por historia. Pero esta base ahorra muchas horas. Tiempo estimado: **6 a 8 horas** por persona durante el Sprint 0.

| Recurso                                                          | Qué ver                                                                                                                                                                      | Tiempo |
|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <b>A Tour of Go</b> — go.dev/tour (tiene versión en español)     | Basics (paquetes, variables, funciones, for, if, structs, slices, maps) y Methods and interfaces                                                                             | 3 h    |
| <b>Learn Go with Tests</b> — quii.gitbook.io/learn-go-with-tests | Capítulos Hello World, Integers, Iteration, Arrays and slices, Structs/methods/interfaces, Maps. <b>Es exactamente TDD en Go:</b> cada capítulo empieza por el test en rojo. | 3 h    |
| <b>GitHub Skills</b> — skills.github.com                         | "Introduction to GitHub" (ramas, commits, Pull Requests)                                                                                                                     | 1 h    |
| <b>htmx.org/examples</b>                                         | Mirar "Click to Edit" y "Active Search" para entender la idea                                                                                                                | 30 min |

### Cómo usar la IA sin que juegue en contra

- La consigna la permite, pero **cada uno debe poder explicar cada línea** de su código en la presentación.
- Pedirle a la IA que **explique** y que proponga el **test primero**; después escribir o ajustar el código uno mismo.
- Nunca pegar código que no compila o que no se entiende: preguntar "¿qué hace esta línea?" hasta entenderla.
- Anotar en docs/ia/registro.md qué se pidió, qué se usó y qué se corrigió (evidencia para la evaluación).

### 3. Go en 10 conceptos (lo mínimo para arrancar)

```
package project                                // 1. Todo archivo pertenece a un paquete (una carpeta = un paquete)

import (                                        // 2. Importar otros paquetes
    "errors"
    "time"
)

var ErrNombreInvalido = errors.New("el nombre debe tener entre 3 y 100 caracteres") // 3. Errores como valores

type Proyecto struct {                        // 4. struct: agrupa datos (como una ficha)
    Nombre string
    Inicio time.Time
    Fin    time.Time
}

func Duracion(p Proyecto) int {               // 5. Función: recibe parámetros y devuelve un resultado
    return int(p.Fin.Sub(p.Inicio).Hours() / 24)
}

func (p Proyecto) EsLargo() bool {           // 6. Método: una función "pegada" a un tipo
    return Duracion(p) > 90
}

func Validar(nombre string) error {           // 7. Si algo puede fallar, se devuelve un error (nil = todo bien)
    if len(nombre) < 3 {
        return ErrNombreInvalido
    }
    return nil
}

func SumarPuntos(puntos []int) int {         // 8. slice: lista de largo variable; for range la recorre
    total := 0                                // := declara y asigna
    for _, p := range puntos {
        total += p
    }
    return total
}

type Repositorio interface {                 // 9. interface: "cualquier cosa que sepa hacer esto"
    Guardar(p Proyecto) error                // (permite usar una base real o una falsa en los tests)
}
```

#### Y el concepto más importante para TDD:

```
package project_test

import (
    "testing"

    "github.com/<org>/software-metrics/internal/domain/project"
)

func TestSumarPuntos(t *testing.T) {         // 10. Un test: función TestXxx en un archivo xxx_test.go
    got := project.SumarPuntos([]int{3, 5, 8})
    if got != 16 {
        t.Errorf("SumarPuntos = %d, se esperaba 16", got) // falla el test con un mensaje claro
    }
}
```

Se ejecuta con `go test ./internal/domain/project/`. Si pasa, dice **ok**; si falla, muestra el mensaje de `t.Errorf`.

## 4. La receta: cómo desarrollar una historia de principio a fin

Esta receta se repite **para cada historia**. Es la cadena de trazabilidad que pide la consigna: Historia → Especificación SDD → Criterios de aceptación → Escenarios BDD → Tests → Código Go.

| Paso                 | Qué hacer                                                                                                                   | Para qué                                                    |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| 1. Tomar la historia | En el tablero, asignarse el issue y moverlo a <b>En progreso</b> .                                                          | Que el equipo sepa en qué está cada uno.                    |
| 2. Crear la rama     | <code>git switch main → git pull → git switch -c feature/HU-01-crear-proyecto</code>                                        | Trabajar sin romper lo de los demás.                        |
| 3. Especificar (SDD) | Copiar <code>specs/_plantilla.md</code> a <code>specs/HU-01-crear-proyecto.md</code> y completar las 8 secciones.           | Pensar reglas, límites y errores <b>antes</b> de programar. |
| 4. Escenarios (BDD)  | Escribir <code>features/hu-01-crear-proyecto.feature</code> con casos normal, alternativo, límite y error.                  | Describir el comportamiento en lenguaje del cliente.        |
| 5. RED               | Escribir <b>un</b> test para un criterio. Correr <code>go test → falla</code> . Commit: <code>test(HU-01): RED - ...</code> | Evidencia de TDD: el test existe antes que el código.       |
| 6. GREEN             | Escribir el código <b>mínimo</b> para que pase. Commit: <code>feat(HU-01): GREEN - ...</code>                               | No programar de más.                                        |
| 7. REFACTOR          | Mejorar nombres o quitar repeticiones con los tests en verde. Commit: <code>refactor(HU-01): ...</code>                     | Código prolijo sin romper nada.                             |
| 8. Repetir 5–7       | Un ciclo por cada criterio de aceptación y caso límite.                                                                     | Cubrir toda la spec.                                        |
| 9. Conectar capas    | Servicio → repositorio → handler → plantilla. Correr los escenarios con <code>go test ./features/...</code>                 | Que la funcionalidad se pueda usar.                         |
| 10. Pull Request     | <code>git push -u origin feature/HU-01-crear-proyecto → gh pr create → completar la plantilla</code>                        | Revisión de un compañero y CI automático.                   |
| 11. Revisión y merge | El revisor aprueba, se hace <b>merge commit</b> (nunca squash) y se mueve la tarjeta a <b>Hecho</b> .                       | El deploy a Vercel es automático.                           |

### Ejemplo de un ciclo en la terminal

```
git add internal/domain/project/project_test.go
git commit -m "test(HU-01): RED - nombre con menos de 3 caracteres es inválido"
go test ./internal/domain/project/          # FAIL (esperado)
# ... escribir el código mínimo ...
go test ./internal/domain/project/          # ok
git add internal/domain/project/project_test.go
git commit -m "feat(HU-01): GREEN - valida largo del nombre"
```

## 4.1 Plantilla de especificación SDD (specs/\_plantilla.md)

```
# SPEC-XX · <Título de la historia>

**Historia:** HU-XX · **Responsable:** <nombre> · **Sprint:** N · **Estado:** Borrador | Revisada

## 1. Objetivo
Qué problema resuelve y para quién (1 o 2 oraciones).

## 2. Entradas
| Dato | Tipo | Obligatorio | Validación |

## 3. Salidas esperadas
Qué devuelve o muestra el sistema cuando todo sale bien.

## 4. Reglas de negocio
- RN1: ...
- RN2: ...

## 5. Restricciones
Técnicas o de negocio (por ejemplo: "cálculo puro, sin acceso a la base").

## 6. Casos límite
Valores en el borde: vacío, cero, máximo permitido, un solo elemento, fechas iguales...

## 7. Condiciones de error
| Situación | Error | Mensaje al usuario |

## 8. Criterios de aceptación
- CA-XX.1 ...
- CA-XX.2 ...

## Trazabilidad
Feature: features/hu-xx-....feature · Tests: internal/.../xxx_test.go · PR: #
```

## 5. Ejemplo de referencia completo: HU-01 Crear proyecto

Este ejemplo recorre **todas las capas**. Es de Juan Pablo, pero **los tres lo usan como modelo** para sus historias: sólo cambian los nombres y las reglas. Convención: paquetes con nombre corto en inglés (project, sprint), tipos y funciones del dominio en español (Proyecto, Nuevo).

### 5.1 Escenarios BDD (features/hu-01-crear-proyecto.feature)

```
# language: es
@HU-01
Característica: Crear proyecto
  Como Agile Enabler
  Quiero crear un proyecto con nombre y fechas
  Para comenzar a gestionarlo

  Escenario: Alta exitosa (CA-01.3)
    Cuando creo el proyecto "Metrics App" del "2026-10-05" al "2026-11-05"
    Entonces el proyecto queda en estado "Planificado"

  Escenario: Nombre repetido (CA-01.1)
    Dado que existe el proyecto "Metrics App"
    Cuando creo el proyecto "metrics app" del "2026-10-05" al "2026-11-05"
    Entonces veo el error "ya existe un proyecto con ese nombre"

  Escenario: Fechas invertidas (CA-01.2)
    Cuando creo el proyecto "Metrics App" del "2026-11-05" al "2026-10-05"
    Entonces veo el error "la fecha de fin debe ser posterior a la de inicio"
```

## 5.2 Dominio: la regla de negocio (internal/domain/project/project.go)

```
package project

import (
    "errors"
    "strings"
    "time"
)

// Estado de un proyecto.
type Estado string

const (
    EstadoPlanificado Estado = "Planificado"
    EstadoEnCurso      Estado = "En curso"
    EstadoFinalizado   Estado = "Finalizado"
)

var (
    ErrNombreInvalido = errors.New("el nombre debe tener entre 3 y 100 caracteres")
    ErrFechasInvalidas = errors.New("la fecha de fin debe ser posterior a la de inicio")
)

type Proyecto struct {
    ID          int64
    Nombre      string
    Descripcion string
    Inicio      time.Time
    Fin         time.Time
    Estado      Estado
}

// Nuevo crea un proyecto válido o devuelve el error de la primera regla que no se cumple.
func Nuevo(nombre, descripcion string, inicio, fin time.Time) (Proyecto, error) {
    nombre = strings.TrimSpace(nombre)
    if n := len([]rune(nombre)); n < 3 || n > 100 {
        return Proyecto{}, ErrNombreInvalido
    }
    if !fin.After(inicio) {
        return Proyecto{}, ErrFechasInvalidas
    }
    return Proyecto{
        Nombre:      nombre,
        Descripcion: strings.TrimSpace(descripcion),
        Inicio:      inicio,
        Fin:         fin,
        Estado:      EstadoPlanificado,
    }, nil
}
```

- `len([]rune(nombre))` cuenta **letras**, no bytes: "Gestión" tiene 7 letras pero 8 bytes por la tilde.
- Los errores son **variables con nombre** (`ErrNombreInvalido`): los tests y las pantallas pueden reconocerlos con `errors.Is`.
- No hay nada de web ni de base de datos acá: por eso el test es trivial de escribir.

### 5.3 Test del dominio, escrito ANTES (internal/domain/project/project\_test.go)

```
package project_test

import (
    "errors"
    "testing"
    "time"

    "github.com/<org>/software-metrics/internal/domain/project"
)

func TestNuevo(t *testing.T) {
    inicio := time.Date(2026, 10, 5, 0, 0, 0, 0, time.UTC)
    fin := inicio.AddDate(0, 1, 0)

    casos := []struct {
        caso      string
        nombre     string
        inicio, fin time.Time
        errorEsperado error
    }{
        {"datos válidos (CA-01.3)", "Metrics App", inicio, fin, nil},
        {"nombre vacío (CA-01.1)", " ", inicio, fin, project.ErrNombreInvalido},
        {"nombre de 2 letras (límite)", "AB", inicio, fin, project.ErrNombreInvalido},
        {"fin igual al inicio (límite)", "Metrics App", inicio, inicio, project.ErrFechasInvalidas},
        {"fin antes del inicio (CA-01.2)", "Metrics App", fin, inicio, project.ErrFechasInvalidas},
    }
    for _, c := range casos {
        t.Run(c.caso, func(t *testing.T) {
            p, err := project.Nuevo(c.nombre, "", c.inicio, c.fin)
            if !errors.Is(err, c.errorEsperado) {
                t.Fatalf("error = %v, se esperaba %v", err, c.errorEsperado)
            }
            if err == nil && p.Estado != project.EstadoPlanificado {
                t.Errorf("estado = %q, se esperaba %q", p.Estado, project.EstadoPlanificado)
            }
        })
    }
}
```

Esto se llama **test de tabla**: cada fila es un caso. Agregar un caso nuevo es agregar una línea. Cada nombre de caso cita el criterio de aceptación que prueba (**trazabilidad**).

### 5.4 Servicio: reglas que necesitan datos guardados (internal/service/proyectos.go)

"El nombre debe ser único" no se puede verificar en el dominio porque hay que consultar la base. Eso va en el servicio, que depende de una **interfaz** de repositorio (no de Supabase directamente).

```

package service

import (
    "context"
    "errors"
    "fmt"
    "time"

    "github.com/<org>/software-metrics/internal/domain/project"
)

var ErrNombreDuplicado = errors.New("ya existe un proyecto con ese nombre")

// ProyectoRepo es lo que el servicio necesita de la base de datos.
type ProyectoRepo interface {
    ExisteNombre(ctx context.Context, nombre string) (bool, error)
    Guardar(ctx context.Context, p *project.Proyecto) error
}

type Proyectos struct {
    repo ProyectoRepo
}

func NuevoProyectos(repo ProyectoRepo) *Proyectos {
    return &Proyectos{repo: repo}
}

func (s *Proyectos) Crear(ctx context.Context, nombre, descripcion string, inicio, fin time.Time) (project.Proyecto, error) {
    p, err := project.Nuevo(nombre, descripcion, inicio, fin)
    if err != nil {
        return project.Proyecto{}, err
    }
    existe, err := s.repo.ExisteNombre(ctx, p.Nombre)
    if err != nil {
        return project.Proyecto{}, fmt.Errorf("verificando nombre: %w", err)
    }
    if existe {
        return project.Proyecto{}, ErrNombreDuplicado
    }
    if err := s.repo.Guardar(ctx, &p); err != nil {
        return project.Proyecto{}, fmt.Errorf("guardando proyecto: %w", err)
    }
    return p, nil
}

```

- `context.Context` viaja en cada llamada: permite cancelar la operación si el usuario cierra la página. Siempre es el primer parámetro.
- `fmt.Errorf("...: %w", err)` **envuelve** el error agregando contexto sin perder el original (`errors.Is` lo sigue encontrando).

## 5.5 Repositorio falso para los tests (internal/store/memory/proyectos.go)

```
package memory

import (
    "context"
    "strings"
    "sync"

    "github.com/<org>/software-metrics/internal/domain/project"
)

// Proyectos guarda en memoria: sirve para tests y escenarios BDD sin base de datos.
type Proyectos struct {
    mu      sync.Mutex
    datos []project.Proyecto
}

func (r *Proyectos) ExisteNombre(_ context.Context, nombre string) (bool, error) {
    r.mu.Lock()
    defer r.mu.Unlock()
    for _, p := range r.datos {
        if strings.EqualFold(p.Nombre, nombre) { // compara sin distinguir mayúsculas
            return true, nil
        }
    }
    return false, nil
}

func (r *Proyectos) Guardar(_ context.Context, p *project.Proyecto) error {
    r.mu.Lock()
    defer r.mu.Unlock()
    p.ID = int64(len(r.datos) + 1)
    r.datos = append(r.datos, *p)
    return nil
}
```

### Test del servicio (internal/service/proyectos\_test.go)

```
func TestCrear_NombreDuplicado(t *testing.T) {
    svc := service.NuevoProyectos(&memory.Proyectos{})
    ctx := context.Background()
    inicio := time.Date(2026, 10, 5, 0, 0, 0, 0, time.UTC)
    fin := inicio.AddDate(0, 1, 0)

    if _, err := svc.Crear(ctx, "Metrics App", "", inicio, fin); err != nil {
        t.Fatalf("primer alta: error inesperado %v", err)
    }
    _, err := svc.Crear(ctx, "metrics app", "", inicio, fin) // mismo nombre en minúsculas
    if !errors.Is(err, service.ErrNombreDuplicado) {
        t.Fatalf("error = %v, se esperaba ErrNombreDuplicado", err)
    }
}
```

## 5.6 Repositorio real con Supabase (internal/store/postgres/proyectos.go)

### Migración: crea la tabla (se versiona en Git como el código)

```
-- migrations/0001_proyectos.sql
-- +goose Up
CREATE TABLE proyectos (
  id          BIGSERIAL PRIMARY KEY,
  nombre      TEXT NOT NULL,
  descripcion TEXT NOT NULL DEFAULT '',
  fecha_inicio DATE NOT NULL,
  fecha_fin   DATE NOT NULL,
  estado      TEXT NOT NULL DEFAULT 'Planificado',
  creado_en   TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (fecha_fin > fecha_inicio)
);
CREATE UNIQUE INDEX proyectos_nombre_unico ON proyectos (lower(nombre));

-- +goose Down
DROP TABLE proyectos;
```

```
package postgres

type Proyectos struct {
  db *pgxpool.Pool
}

func (r *Proyectos) ExisteNombre(ctx context.Context, nombre string) (bool, error) {
  var existe bool
  err := r.db.QueryRow(ctx,
    `SELECT EXISTS (SELECT 1 FROM proyectos WHERE lower(nombre) = lower($1))`, nombre,
  ).Scan(&existe)
  return existe, err
}

func (r *Proyectos) Guardar(ctx context.Context, p *project.Proyecto) error {
  return r.db.QueryRow(ctx,
    `INSERT INTO proyectos (nombre, descripcion, fecha_inicio, fecha_fin, estado)
    VALUES ($1, $2, $3, $4, $5) RETURNING id`,
    p.Nombre, p.Descripcion, p.Inicio, p.Fin, string(p.Estado),
  ).Scan(&p.ID)
}
```

\$1, \$2... son **parámetros**: nunca armar SQL concatenando texto del usuario (evita inyección SQL). El servicio no nota la diferencia entre el repositorio en memoria y este: los dos cumplen ProyectoRepo.

## 5.7 Handler y pantalla (internal/http/proyectos.go y web/templates/proyecto\_form.html)

```
func (h *Handler) CrearProyecto(w http.ResponseWriter, r *http.Request) {
    inicio, errI := time.Parse("2006-01-02", r.FormValue("inicio")) // formato de <input type="date">
    fin, errF := time.Parse("2006-01-02", r.FormValue("fin"))
    if errI != nil || errF != nil {
        h.render(w, "proyecto_form", map[string]any{"Error": "Ingresá fechas válidas"})
        return
    }
    p, err := h.proyectos.Crear(r.Context(), r.FormValue("nombre"), r.FormValue("descripcion"), inicio, fin)
    if err != nil {
        h.render(w, "proyecto_form", map[string]any{"Error": mensajeUsuario(err)})
        return
    }
    w.Header().Set("HX-Redirect", fmt.Sprintf("/proyectos/%d", p.ID)) // HTMX navega al proyecto creado
}

// mensajeUsuario muestra los errores de negocio tal cual y oculta los técnicos.
func mensajeUsuario(err error) string {
    switch {
    case errors.Is(err, project.ErrNombreInvalido),
        errors.Is(err, project.ErrFechasInvalidas),
        errors.Is(err, service.ErrNombreDuplicado):
        return err.Error()
    default:
        log.Printf("error inesperado: %v", err)
        return "Ocurrió un error inesperado. Intentá de nuevo."
    }
}
```

```
{{define "proyecto_form"}}
<form hx-post="/proyectos" hx-swap="outerHTML" class="card card-body gap-3">
    {{with .Error}}<div class="alert alert-danger mb-0">{{.}}</div>{{end}}
    <div>
        <label class="form-label" for="nombre">Nombre</label>
        <input id="nombre" name="nombre" class="form-control" required minlength="3" maxlength="100">
    </div>
    <div>
        <label class="form-label" for="descripcion">Descripción</label>
        <textarea id="descripcion" name="descripcion" class="form-control" rows="2"></textarea>
    </div>
    <div class="row g-3">
        <div class="col"><label class="form-label" for="inicio">Inicio</label>
        <input id="inicio" type="date" name="inicio" class="form-control" required></div>
        <div class="col"><label class="form-label" for="fin">Fin</label>
        <input id="fin" type="date" name="fin" class="form-control" required></div>
    </div>
    <button class="btn btn-primary">Crear proyecto</button>
</form>
{{end}}
```

- `hx-post="/proyectos"` envía el formulario sin recargar; `hx-swap="outerHTML"` reemplaza el formulario por la respuesta (el mismo formulario con el mensaje de error, si lo hubo).
- Las clases `form-control`, `btn btn-primary`, `card` son de Bootstrap: dan el estilo sin escribir CSS.
- La validación en el HTML (`required`, `minlength`) es sólo comodidad: **la regla real está en Go** y es la que se testea.

## 5.8 Pasos de godog que conectan el .feature con el servicio (features/steps\_test.go)

```

type mundo struct {
    svc      *service.Proyectos
    proyecto project.Proyecto
    err      error
}

func (m *mundo) existeElProyecto(nombre string) error {
    _, err := m.svc.Crear(context.Background(), nombre, "", fecha("2026-10-05"), fecha("2026-11-05"))
    return err
}

func (m *mundo) creoElProyecto(nombre, inicio, fin string) error {
    m.proyecto, m.err = m.svc.Crear(context.Background(), nombre, "", fecha(inicio), fecha(fin))
    return nil // el error se guarda: lo verifica el paso "Entonces"
}

func (m *mundo) quedaEnEstado(estado string) error {
    if m.err != nil || string(m.proyecto.Estado) != estado {
        return fmt.Errorf("estado = %q (error %v), se esperaba %q", m.proyecto.Estado, m.err, estado)
    }
    return nil
}

func (m *mundo) veoElError(mensaje string) error {
    if m.err == nil || m.err.Error() != mensaje {
        return fmt.Errorf("se esperaba el error %q y se obtuvo %v", mensaje, m.err)
    }
    return nil
}

func fecha(s string) time.Time {
    t, _ := time.Parse("2006-01-02", s)
    return t
}

func InitializeScenario(sc *godog.ScenarioContext) {
    m := &mundo{}
    sc.Before(func(ctx context.Context, _ *godog.Scenario) (context.Context, error) {
        *m = mundo{svc: service.NuevoProyectos(&memory.Proyectos{})} // cada escenario arranca limpio
        return ctx, nil
    })
    sc.Step(`^que existe el proyecto "([^"]*)"`, m.existeElProyecto)
    sc.Step(`^creo el proyecto "([^"]*)" del "([^"]*)" al "([^"]*)"`, m.creoElProyecto)
    sc.Step(`^el proyecto queda en estado "([^"]*)"`, m.quedaEnEstado)
    sc.Step(`^veo el error "([^"]*)"`, m.veoElError)
}

func TestFeatures(t *testing.T) {
    suite := godog.TestSuite{
        ScenarioInitializer: InitializeScenario,
        Options:             &godog.Options{Format: "pretty", Paths: []string{"."}, TestingT: t},
    }
    if suite.Run() != 0 {
        t.Fatal("fallaron escenarios BDD")
    }
}

```

Cada frase del .feature se asocia con una función mediante una **expresión regular**: "`"([^"]*)"`" captura el texto entre comillas y lo pasa como parámetro. Así el escenario escrito en español **se ejecuta como un test**.

## 6. Guía individual de cada integrante

Cada guía dice **qué construye**, **para qué sirve** (qué problema del usuario resuelve), **qué hace en cada sprint**, qué conceptos le conviene aprender y un **ejemplo de arranque** con su test. Las historias y criterios completos están en el Plan de trabajo.

### 6.1 Juan Pablo — Proyectos, Sprints, Reportes + Agile Enabler

#### Qué construí y para qué

- **Proyectos e integrantes (E1)**: es la puerta de entrada; sin proyecto no hay backlog, sprints ni métricas.
- **Sprints (E3)**: el corazón de Scrum: crear el sprint con su objetivo, comprometer historias, marcarlas terminadas y **cerrarlo**, que es el momento en que se "congelan" los números que después usan las métricas de Carolina.
- **Reportes y PDF (E9)**: lo que el equipo le entrega al cliente en cada Review; reutiliza los cálculos de Carolina.
- **Agile Enabler**: que el proceso Scrum se cumpla y quede registrado (vale el 20 % de la nota: gestión del proyecto).

| Sprint | Qué hacés                                                                                                                                                 |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0      | Tablero en GitHub Projects, plantillas, esqueleto del proyecto con Claude (TEC-01), CI (TEC-02), carga de issues E1/E3/E9, organizar la primera Planning. |
| 1      | HU-01 Crear proyecto (ejemplo de la sección 5), HU-03 Integrantes, HU-11 Crear sprint, HU-12 Asignar historias, HU-13 Completadas.                        |
| 2      | HU-02 Modificar proyecto, HU-04 Estado del proyecto, HU-14 <b>Cerrar sprint</b> (ejemplo abajo), HU-15 Sprints anteriores + sus pantallas.                |
| 3      | HU-37 Reporte de sprint y HU-38 Reporte de proyecto (en pantalla), empezar la presentación.                                                               |
| 4      | HU-39 Exportar a PDF, compilar backlogs y actas, guion de la demo final.                                                                                  |

**Conceptos a aprender:** structs y métodos con receptor puntero (func (s \*Sprint)), manejo de fechas con time, máquinas de estado simples, html/template y ciclos {{range}} para listas.

## Ejemplo de arranque: cerrar un sprint (HU-14)

```
package sprint

type Estado string

const (
    EstadoPlanificado Estado = "Planificado"
    EstadoActivo       Estado = "Activo"
    EstadoCerrado      Estado = "Cerrado"
)

var ErrSprintNoActivo = errors.New("sólo se puede cerrar el sprint activo")

// HistoriaDelSprint es lo mínimo que el sprint necesita saber de cada historia.
type HistoriaDelSprint struct {
    ID          int64
    StoryPoints int
    Completada  bool
}

type Sprint struct {
    ID          int64
    Numero      int
    Objetivo    string
    Inicio, Fin  time.Time
    Estado      Estado
    SPPlanificados int
    SPCompletados int
}

// Cerrar guarda la foto de story points (CA-14.3) y devuelve las historias que vuelven al backlog (CA-14.2).
func (s *Sprint) Cerrar(historias []HistoriaDelSprint) ([]int64, error) {
    if s.Estado != EstadoActivo {
        return nil, ErrSprintNoActivo // CA-14.1 y CA-14.4
    }
    var pendientes []int64
    s.SPPlanificados, s.SPCompletados = 0, 0
    for _, h := range historias {
        s.SPPlanificados += h.StoryPoints
        if h.Completada {
            s.SPCompletados += h.StoryPoints
        } else {
            pendientes = append(pendientes, h.ID)
        }
    }
    s.Estado = EstadoCerrado
    return pendientes, nil
}
```

**Test (se escribe primero)**

```
func TestCerrar(t *testing.T) {
    t.Run("guarda la foto de SP y devuelve las pendientes (CA-14.2, CA-14.3)", func(t *testing.T) {
        s := sprint.Sprint{Numero: 1, Estado: sprint.EstadoActivo}
        historias := []sprint.HistoriaDelSprint{
            {ID: 1, StoryPoints: 5, Completada: true},
            {ID: 2, StoryPoints: 3},
            {ID: 3, StoryPoints: 8, Completada: true},
        }
        pendientes, err := s.Cerrar(historias)
        if err != nil {
            t.Fatalf("error inesperado: %v", err)
        }
        if s.SPCompletados != 13 || s.SPPlanificados != 16 {
            t.Errorf("SP completados/planificados = %d/%d, se esperaba 13/16", s.SPCompletados, s.SPPlanificados)
        }
        if !slices.Equal(pendientes, []int64{2}) {
            t.Errorf("pendientes = %v, se esperaba [2]", pendientes)
        }
        if s.Estado != sprint.EstadoCerrado {
            t.Errorf("estado = %q, se esperaba Cerrado", s.Estado)
        }
    })
    t.Run("un sprint cerrado no se vuelve a cerrar (CA-14.4)", func(t *testing.T) {
        s := sprint.Sprint{Estado: sprint.EstadoCerrado}
        if _, err := s.Cerrar(nil); !errors.Is(err, sprint.ErrSprintNoActivo) {
            t.Fatalf("error = %v, se esperaba ErrSprintNoActivo", err)
        }
    })
}
```

**Para pensar vos (HU-11):** dos sprints se superponen si ninguno termina antes de que empiece el otro. Escribí primero los tests: sprints consecutivos (no se superponen), uno dentro del otro, y el caso límite en que uno termina el mismo día que empieza el otro.

**Tu rol de Agile Enabler en sprints de una semana**

| Ceremonia       | Cuándo y cuánto                                 | Qué hacés                                                                                                                                   | Registro en el repo             |
|-----------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| Sprint Planning | Lunes, 1 h                                      | Repasar velocidad, elegir historias con el equipo, re-estimar con Planning Poker, fijar el Sprint Goal, asignar la iteración en el tablero. | docs/scrum/sprint-N/planning.md |
| Daily           | Lunes a viernes, 10 min (puede ser por mensaje) | Cada uno responde: qué hice, qué voy a hacer, qué me bloquea. Vos destrabás los bloqueos.                                                   | docs/scrum/sprint-N/dailies.md  |
| Sprint Review   | Fin de la semana (o la clase siguiente), 30 min | Demo <b>en Vercel</b> de lo terminado; anotar el feedback del profesor; nuevas historias al backlog.                                        | review.md                       |
| Retrospective   | Después de la Review, 30 min                    | Qué salió bien / qué mejorar / acciones con responsable. Revisar si se cumplieron las acciones anteriores.                                  | retro.md (acta)                 |

**Plantilla de acta de retrospectiva (docs/scrum/sprint-N/retro.md)**

```
# Retrospectiva · Sprint N · <fecha>
Participantes: López, Piastrellini, Bravo · Facilitador: Juan Pablo

## Métricas del sprint
SP comprometidos: __ · SP completados: __ · Velocidad: __ · Defectos abiertos: __

## ¿Qué salió bien?
- ...
## ¿Qué podemos mejorar?
- ...
## Acciones (con responsable y fecha)
| Acción | Responsable | Para cuándo |
## Seguimiento de acciones del sprint anterior
- [x] ... / [ ] ...
```

## 6.2 Mariano — Product Backlog, Planning Poker, base de datos y login

### Qué construís y para qué

- **Product Backlog (E2):** la lista ordenada de todo lo que hay que construir. Sin historias bien cargadas no se puede planificar ni medir.
- **Planning Poker (E4):** el equipo estima en conjunto y en secreto para que nadie se deje influir por el primero que habla ("efecto ancla"). Es la funcionalidad más original de la consigna y la que más luce en la demo.
- **Base de datos y login (TEC-03, TEC-05):** donde viven los datos de todos y quién es quién (necesario para votar).

| Sprint | Qué hacés                                                                                                                                           |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 0      | Crear los proyectos de Supabase (dev y prod), proponer el diagrama de datos, plantilla SDD, cargar issues de E2/E4.                                 |
| 1      | TEC-03 Migraciones y repositorios, HU-05 Crear historia, HU-06 Editar/eliminar, HU-07 Criterios de aceptación, HU-08 Estados, HU-16 Estimar con SP. |
| 2      | TEC-05 Login, HU-09 Priorizar, HU-10 Filtrar, HU-17 Abrir sesión, HU-18 <b>Votar oculto</b> (ejemplo abajo), HU-19 Revelar + pantallas.             |
| 3      | HU-20 <b>Detectar diferencias</b> (ejemplo abajo), HU-21 Nueva ronda, HU-22 Registrar acordada, TEC-06 Tests de integración.                        |
| 4      | Documentación técnica (arquitectura, datos, cómo correr y desplegar), auditoría de trazabilidad, corrección de defectos.                            |

**Conceptos a aprender:** SQL básico (CREATE TABLE, INSERT, SELECT, JOIN), `map` en Go, encapsulamiento (campos en minúscula = privados del paquete), máquinas de estado (Pendiente → En sprint → En progreso → Hecho), polling con HTMX.

## Ejemplo de arranque: votos ocultos del Planning Poker (HU-18, HU-19)

```
package estimation

var EscalaFibonacci = []int{0, 1, 2, 3, 5, 8, 13, 21}

var (
    ErrFueraDeEscala = errors.New("el valor no pertenece a la escala")
    ErrNoParticipa   = errors.New("el integrante no participa de la sesión")
    ErrRondaRevelada = errors.New("la ronda ya fue revelada")
    ErrSinVotos      = errors.New("no se puede revelar una ronda sin votos")
)

type Ronda struct {
    Numero    int
    Revelada  bool
    votos     map[int64]int // integranteID -> valor. En minúscula: nadie fuera del paquete puede leerlo
}

type Sesión struct {
    HistoriaID int64
    Participantes []int64
    Rondas     []*Ronda
}

func NuevaSesión(historiaID int64, participantes []int64) *Sesión {
    primera := &Ronda{Numero: 1, votos: map[int64]int{}}
    return &Sesión{HistoriaID: historiaID, Participantes: participantes, Rondas: []*Ronda{primera}}
}

func (s *Sesión) rondaActual() *Ronda { return s.Rondas[len(s.Rondas)-1] }

// Votar registra o reemplaza el voto del integrante en la ronda actual (CA-18.1).
func (s *Sesión) Votar(integranteID int64, valor int) error {
    if !slices.Contains(s.Participantes, integranteID) {
        return ErrNoParticipa
    }
    if !slices.Contains(EscalaFibonacci, valor) {
        return ErrFueraDeEscala // CA-18.4
    }
    r := s.rondaActual()
    if r.Revelada {
        return ErrRondaRevelada
    }
    r.votos[integranteID] = valor
    return nil
}

// QuienesVotaron devuelve sólo quiénes votaron, nunca los valores (CA-18.2).
func (s *Sesión) QuienesVotaron() []int64 {
    ids := make([]int64, 0, len(s.rondaActual().votos))
    for id := range s.rondaActual().votos {
        ids = append(ids, id)
    }
    slices.Sort(ids)
    return ids
}

// Revelar cierra la ronda y recién ahí entrega los votos (CA-19.1, CA-19.3).
func (s *Sesión) Revelar() (map[int64]int, error) {
    r := s.rondaActual()
    if len(r.votos) == 0 {
        return nil, ErrSinVotos
    }
    r.Revelada = true
    return maps.Clone(r.votos), nil // copia: quien la reciba no puede modificar la original
}
```

- El "secreto" está garantizado por el **lenguaje**: votos está en minúscula, así que el handler sólo puede usar QuienesVotaron() antes de revelar. Es un gran argumento para la defensa oral.
- Para guardar en Supabase: cada voto es una fila de la tabla votos (sesion\_id, ronda, integrante\_id, valor); se agrega una función RestaurarSesión(...) que reconstruye la sesión desde la base.

- Como Vercel no mantiene conexiones abiertas, la pantalla pregunta cada 2 segundos quién votó: `<div hx-get="/poker/12/estado" hx-trigger="every 2s">`.

### Segundo ejemplo: detectar diferencias (HU-20)

```
// HayDiscrepancia indica si el voto más alto y el más bajo están a más de una posición de distancia
// en la escala. Ejemplo: 3 y 5 son vecinos (no hay discrepancia); 2 y 8 sí la tienen.
func HayDiscrepancia(votos map[int64]int) bool {
    primero := true
    var menor, mayor int
    for _, v := range votos {
        pos := slices.Index(EscalaFibonacci, v)
        if primero {
            menor, mayor, primero = pos, pos, false
            continue
        }
        menor, mayor = min(menor, pos), max(mayor, pos)
    }
    return mayor-menor > 1
}
```

### Test

```
func TestHayDiscrepancia(t *testing.T) {
    casos := []struct {
        caso string
        votos map[int64]int
        want bool
    }{
        {"todos iguales", map[int64]int{1: 5, 2: 5, 3: 5}, false},
        {"valores vecinos en la escala", map[int64]int{1: 3, 2: 5}, false},
        {"valores lejanos (CA-20.1)", map[int64]int{1: 2, 2: 8}, true},
        {"un solo voto (límite)", map[int64]int{1: 13}, false},
        {"sin votos (límite)", map[int64]int{}, false},
    }
    for _, c := range casos {
        t.Run(c.caso, func(t *testing.T) {
            if got := estimation.HayDiscrepancia(c.votos); got != c.want {
                t.Errorf("HayDiscrepancia(%v) = %v, se esperaba %v", c.votos, got, c.want)
            }
        })
    }
}
```

### Cómo va a funcionar el login (TEC-05, Sprint 2)

1. La pantalla de login envía email y contraseña a Go; Go se los pasa a **Supabase Auth**, que devuelve un *\*token\** (JWT) si son correctos.
2. Go guarda el token en una **cookie segura** (HttpOnly) del navegador.
3. En cada pedido, un **middleware** (función que corre antes de todos los handlers) valida el token; si es inválido o venció, redirige al login.
4. Con el usuario identificado se busca su integrante: así sabemos quién vota o carga horas. Lo armamos juntos cuando llegue el Sprint 2.

## 6.3 Carolina — Esfuerzo, Defectos, Métricas, Dashboard y diseño

### Qué construís y para qué

- **Esfuerzo (E5):** registrar las horas reales permite comparar lo estimado con lo trabajado y mejorar las estimaciones.
- **Defectos (E6):** medir la calidad: cuántos errores aparecen, qué tan graves son y en qué sprint se resuelven.
- **Métricas (E7):** convierten los datos de los tres en números útiles (velocidad, desviación, % completado). Son cálculos puros: **el lugar ideal para lucir TDD.**
- **Dashboard y diseño (E8, TEC-07):** la cara de la app: lo primero que ve el profesor en la demo.

| Sprint | Qué hacés                                                                                                                                                       |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0      | Proyecto en Vercel (prueba "hola mundo"), plantilla BDD con godog andando, docs/metricas.md con cada fórmula, cargar issues de E5–E8.                           |
| 1      | TEC-04 Deploy, HU-23 Horas estimadas, HU-24 <b>Registrar esfuerzo</b> (ejemplo abajo), HU-30 SP planificados/completados, HU-31 Velocidad, HU-32 % completadas. |
| 2      | TEC-07 <b>Diseño base</b> (primero, porque los demás lo usan), HU-25 Tareas, HU-26 Consultar horas, HU-27/28/29 Defectos.                                       |
| 3      | HU-33 <b>Desviación de horas</b> (ejemplo abajo), HU-34 Defectos detectados/resueltos, HU-35 Panel, HU-36 <b>Gráficos</b> .                                     |
| 4      | Manual de usuario con capturas, informe de métricas y cobertura, corrección de defectos.                                                                        |

**Conceptos a aprender:** números decimales (float64) y división por cero, fechas con time, JSON (encoding/json), plantillas con herencia ({{template}}), grilla y componentes de Bootstrap, Chart.js.

### Ejemplo de arranque: validar un registro de horas (HU-24)

```
package effort

var (
    ErrHorasInvalidas = errors.New("las horas deben ser mayores a 0 y como máximo 24")
    ErrFechaFutura    = errors.New("no se puede registrar esfuerzo en una fecha futura")
    ErrExcedeDia       = errors.New("el total del día supera las 24 horas")
)

type Registro struct {
    ID             int64
    IntegranteID   int64
    HistoriaID     int64
    Fecha          time.Time
    Actividad      string
    Horas          float64
}

// Validar controla un registro nuevo. hoy se recibe como parámetro (y no con time.Now())
// para que los tests den siempre el mismo resultado, sin importar el día en que se corran.
func Validar(r Registro, horasYaCargadasEseDia float64, hoy time.Time) error {
    if r.Horas <= 0 || r.Horas > 24 {
        return ErrHorasInvalidas // CA-24.1
    }
    if r.Fecha.After(hoy) {
        return ErrFechaFutura // CA-24.2
    }
    if horasYaCargadasEseDia+r.Horas > 24 {
        return ErrExcedeDia // CA-24.1 (total diario)
    }
    return nil
}
```

**Test**

```
func TestValidar(t *testing.T) {
    hoy := time.Date(2026, 10, 7, 0, 0, 0, 0, time.UTC)
    casos := []struct {
        caso      string
        horas      float64
        fecha      time.Time
        yaCargadas float64
        want       error
    }{
        {"registro normal", 4, hoy, 0, nil},
        {"hora y media (decimales)", 1.5, hoy, 0, nil},
        {"cero horas (límite)", 0, hoy, 0, effort.ErrHorasInvalidas},
        {"25 horas", 25, hoy, 0, effort.ErrHorasInvalidas},
        {"fecha de mañana", 2, hoy.AddDate(0, 0, 1), 0, effort.ErrFechaFutura},
        {"completa exactamente 24 h (límite)", 4, hoy, 20, nil},
        {"supera 24 h en el día", 5, hoy, 20, effort.ErrExcedeDia},
    }
    for _, c := range casos {
        t.Run(c.caso, func(t *testing.T) {
            r := effort.Registro{Horas: c.horas, Fecha: c.fecha}
            if err := effort.Validar(r, c.yaCargadas, hoy); !errors.Is(err, c.want) {
                t.Errorf("error = %v, se esperaba %v", err, c.want)
            }
        })
    }
}
```

**Segundo ejemplo: desviación entre horas estimadas y reales (HU-33)**

```
// Desviacion compara horas estimadas y reales.
// Si no hubo estimación, el porcentaje no se puede calcular: ok = false (CA-33.2).
func Desviacion(estimadas, reales float64) (absoluta, porcentaje float64, ok bool) {
    absoluta = reales - estimadas
    if estimadas == 0 {
        return absoluta, 0, false
    }
    return absoluta, absoluta / estimadas * 100, true
}
```

Casos de test sugeridos: estimadas 10 y reales 12 → +2 h y +20 %; estimadas 10 y reales 8 → -2 h y -20 %; estimadas 0 y reales 5 → +5 h y porcentaje "no aplicable". El caso de **división por cero** es justo lo que los profesores buscan en "casos límite".

**Tercer ejemplo: diseño base y un gráfico (TEC-07, HU-36)**

**web/templates/base.html — todas las páginas se muestran dentro de este marco**

```
{{define "base"}}<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>{{.Titulo}} · Software Metrics</title>
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
    <script src="https://cdn.jsdelivr.net/npm/htmx.org@2.0.4/dist/htmx.min.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
</head>
<body class="bg-body-tertiary">
    <nav class="navbar navbar-expand bg-primary data-bs-theme="dark">
        <div class="container">
            <a class="navbar-brand" href="/">Software Metrics</a>
            <div class="navbar-nav">
                <a class="nav-link" href="/proyectos">Proyectos</a>
                <a class="nav-link" href="/dashboard">Dashboard</a>
            </div>
        </div>
    </nav>
    <main class="container py-4">{{template "contenido" .}}</main>
</body>
</html>{{end}}
```

**web/templates/dashboard.html — un gráfico de barras con Chart.js**

```

{{define "contenido"}}
<div class="card"><div class="card-body">
  <h5 class="card-title">Velocidad por sprint</h5>
  <canvas id="velocidad"></canvas>
</div></div>
<script>
  fetch("/api/proyectos/{{.ProyectoID}}/velocidad")      // endpoint JSON hecho en Go (y testado)
  .then(r => r.json())
  .then(d => new Chart(document.getElementById("velocidad"), {
    type: "bar",
    data: { labels: d.labels, datasets: [{ label: "SP completados", data: d.data }] }
  }));
</script>
{{end}}

```

**Handler Go que alimenta el gráfico**

```

type serie struct {
  Labels []string `json:"labels"` // "Sprint 1", "Sprint 2", ...
  Data   []int    `json:"data"`   // SP completados de cada sprint
}

func (h *Handler) VelocidadJSON(w http.ResponseWriter, r *http.Request) {
  // ... obtener los sprints cerrados del proyecto con el servicio y armar la serie ...
  w.Header().Set("Content-Type", "application/json")
  if err := json.NewEncoder(w).Encode(s); err != nil {
    http.Error(w, "no se pudo generar la serie", http.StatusInternalServerError)
  }
}

```

Cada página se "parsea" junto con base.html (por ejemplo `template.ParseFS(fs, "templates/base.html", "templates/dashboard.html")`), porque todas definen su propio bloque contenido. Esto queda resuelto en el esqueleto del Sprint 0.

## 7. Los sprints en detalle: qué, para qué y cómo termina cada semana

### Sprint 0 · La Preparación · 28/09 – 04/10

**Qué:** Dejar todo listo para que el lunes 05/10 los tres puedan programar sin trabas.

**Para qué:** Sin esta semana, el Sprint 1 se pierde instalando herramientas y discutiendo cómo nombrar las cosas.

| Integrante | Trabajo del sprint                                                                                                                          |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Juan Pablo | Tablero y plantillas; esqueleto del proyecto con Claude (compila, tests, deploy); CI; issues de E1/E3/E9; convocar reunión con el profesor. |
| Mariano    | Supabase dev/prod; diagrama de datos (lo cierran los tres); plantilla SDD; issues de E2/E4.                                                 |
| Carolina   | Vercel conectado al repo; plantilla BDD con godog; docs/metricas.md; issues de E5-E8.                                                       |

**Además:** Los tres: instalar herramientas (sección 2), cursos de la sección 2.1 y **un primer PR de práctica** cada uno (agregar su nombre al README) para aprender el circuito rama → commit → PR → revisión → merge.

**Cómo sabemos que terminó (demo de la Review):** Tablero con el backlog estimado, repo con esqueleto andando en Vercel, los tres con su primer PR mergeado.

### Sprint 1 · El MVP · 05/10 – 11/10

**Qué:** Primera versión que ya sirve: un proyecto con integrantes, historias estimadas, un sprint con historias, horas cargadas y las primeras métricas.

**Para qué:** El cliente valida temprano que vamos en la dirección correcta, y cada uno aprende la receta completa con historias simples.

| Integrante | Trabajo del sprint                                                                 |
|------------|------------------------------------------------------------------------------------|
| Juan Pablo | HU-01, HU-03, HU-11, HU-12, HU-13 (con pantallas mínimas).                         |
| Mariano    | TEC-03, HU-05, HU-06, HU-07, HU-08, HU-16.                                         |
| Carolina   | TEC-04, HU-23, HU-24, HU-30, HU-31, HU-32 (métricas como funciones puras con TDD). |

**Además:** Planning el lunes 05/10 (primera vez con todas las ceremonias).

**Cómo sabemos que terminó (demo de la Review):** Demo en Vercel: crear proyecto → cargar historias → crear sprint → asignar → completar → ver velocidad y % completado.

### Sprint 2 · La Interfaz · 12/10 – 18/10

**Qué:** Que la app sea usable por cualquiera: login, navegación, pantallas completas y el Planning Poker funcionando entre varios usuarios.

**Para qué:** La consigna evalúa usabilidad (25 % producto funcional) y el Planning Poker es el requisito más distintivo.

| Integrante | Trabajo del sprint                                                              |
|------------|---------------------------------------------------------------------------------|
| Juan Pablo | HU-02, HU-04, HU-14, HU-15 + pantallas de proyectos y sprints.                  |
| Mariano    | TEC-05 login, HU-09, HU-10, HU-17, HU-18, HU-19 + pantallas de backlog y poker. |
| Carolina   | TEC-07 diseño base (primero), HU-25, HU-26, HU-27, HU-28, HU-29 + pantallas.    |

**Además:** Carolina publica el diseño base el martes a más tardar; hasta entonces los demás trabajan en dominio y servicios.

**Cómo sabemos que terminó (demo de la Review):** Demo: dos personas inician sesión desde distintos celulares y estiman una historia con Planning Poker; se cierra un sprint y se registra un defecto.

## Sprint 3 · Funcionalidad y Calidad · 19/10 – 25/10

**Qué:** Completar lo que falta (visualización, rondas de poker, reportes) y robustecer: más tests, errores bien manejados.

**Para qué:** Dos criterios de evaluación (calidad del software 20 % y SDD/BDD/TDD 25 %) se juegan acá.

| Integrante | Trabajo del sprint                                     |
|------------|--------------------------------------------------------|
| Juan Pablo | HU-37, HU-38 reportes en pantalla.                     |
| Mariano    | HU-20, HU-21, HU-22, TEC-06 cobertura mínima en el CI. |
| Carolina   | HU-33, HU-34, HU-35, HU-36 dashboard con gráficos.     |

**Además:** Cada uno revisa la cobertura de sus paquetes (`go test -cover ./...`) y agrega los casos límite que falten.

**Cómo sabemos que terminó (demo de la Review):** Demo: dashboard con gráficos reales, Planning Poker con varias rondas, reporte de un sprint; mostrar la cobertura.

## Sprint 4 · El Cierre · 26/10 – 01/11

**Qué:** Pulir, documentar y preparar la entrega: exportar el informe a PDF y la presentación final.

**Para qué:** Varios entregables son documentos (informe, actas, manual, doc técnica) y la presentación vale el 10 %.

| Integrante | Trabajo del sprint                                                                             |
|------------|------------------------------------------------------------------------------------------------|
| Juan Pablo | HU-39 PDF, compilar backlogs y actas, presentación y guion de la demo de trazabilidad (HU-31). |
| Mariano    | Documentación técnica, auditoría de trazabilidad (toda spec con feature y tests), defectos.    |
| Carolina   | Manual de usuario, informe de métricas y cobertura, defectos.                                  |

**Además:** Congelar el código (versión v1.0.0) al menos dos días antes de la presentación y ensayar la demo.

**Cómo sabemos que terminó (demo de la Review):** Sprint Review final: presentación formal al cliente con demo completa.

## Checklist de fin de sprint (domingo)

- Todas las historias terminadas cumplen la Definition of Done (spec, feature, tests, PR aprobado, en Vercel).
- Lo que no se terminó vuelve al backlog (no se "estira" el sprint).
- Actas de Review y Retro subidas a `docs/scrum/sprint-N/`.
- Tablero actualizado: iteración cerrada y la siguiente preparada para la Planning del lunes.

## 8. Glosario rápido

| Término                | Significado                                                                                               |
|------------------------|-----------------------------------------------------------------------------------------------------------|
| Historia de usuario    | Necesidad contada desde el usuario: Como <rol> quiero <algo> para <beneficio>.                            |
| Story Points (SP)      | Tamaño relativo de una historia (esfuerzo + complejidad + incertidumbre), no horas.                       |
| Velocidad              | SP que el equipo completa por sprint (promedio de los últimos sprints).                                   |
| Criterio de aceptación | Condición verificable que la historia debe cumplir para darse por terminada.                              |
| SDD                    | Specification-Driven Development: escribir la especificación antes del código.                            |
| BDD                    | Behavior-Driven Development: describir el comportamiento con escenarios Dado/Cuando/Entonces ejecutables. |
| TDD                    | Test-Driven Development: test en rojo → código mínimo en verde → refactor.                                |
| Rama (branch)          | Copia paralela del código para trabajar una historia sin afectar main.                                    |
| Commit                 | Foto guardada de los cambios, con un mensaje que explica qué se hizo.                                     |
| Pull Request (PR)      | Pedido para unir una rama a main; ahí se revisa el código y corre el CI.                                  |
| CI                     | Integración continua: GitHub ejecuta tests y linters en cada PR automáticamente.                          |
| Deploy                 | Publicar la app en internet (Vercel lo hace solo al mergear a main).                                      |
| Migración              | Archivo SQL numerado que crea o modifica tablas; se versiona con el código.                               |
| Handler                | Función de Go que atiende un pedido web y devuelve una respuesta.                                         |
| Middleware             | Función que corre antes de los handlers (por ejemplo, verificar el login).                                |
| Cobertura              | Porcentaje del código que ejecutan los tests ( <code>go test -cover</code> ).                             |